

Математика для ЛИНГВИСТОВ



Случайные графы и выделение сообществ

2025/04/10



Случайные графы

План:

1. Что это и зачем нужно
2. Модель Эрдёша — Реньи
3. Модель Барабаши — Альберт
4. Модель маленького мира Уоттса-Строгаца

Что такое случайный граф?

Обычно граф определяется парой (V, E) , где V - множество вершин, а E - множество ребер (упорядоченных пар вершин).

Случайный же граф задается двумя другими сущностями:

- алгоритмом порождения
- распределением вероятностей (зависит от алгоритма)

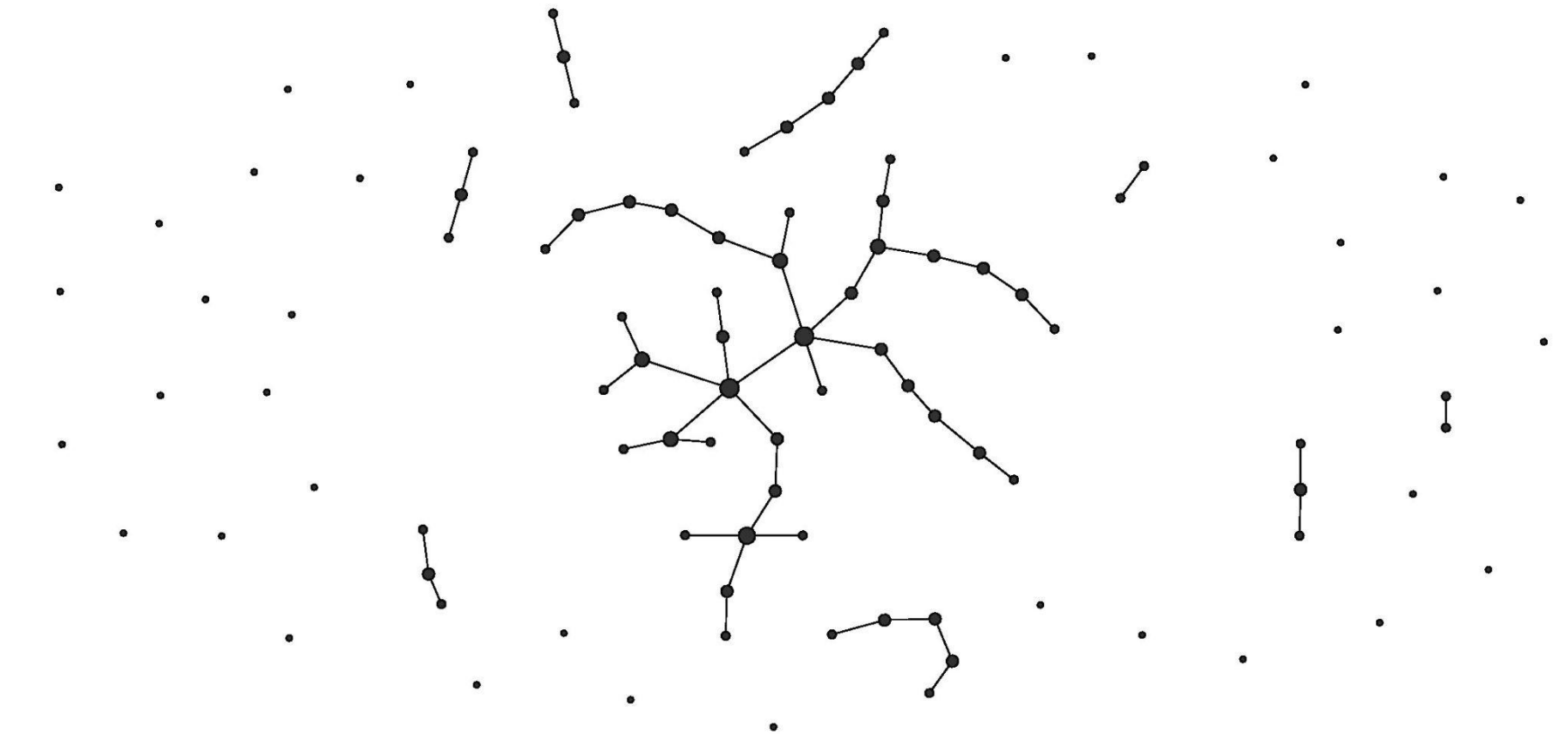
Также существует множество видов случайных деревьев. На них в том числе основан алгоритм машинного обучения Случайный лес.

Как вы считаете, зачем нужны
случайные графы?

Модель Эрдёша — Реньи

Есть две модели с одним названием:

1. $G(n, p)$ - граф на n вершинах, где каждое ребро включается в граф с вероятностью p .
2. $G(n, m)$ - случайный граф из всех графов на n вершинах и m ребрах.



Модель Эрдёша — Реньи

Преимущества модели:

- Простота реализации
- Высокая интерпретируемость из-за независимости ребер
- Есть строгое обоснование для большинства ожидаемых свойств

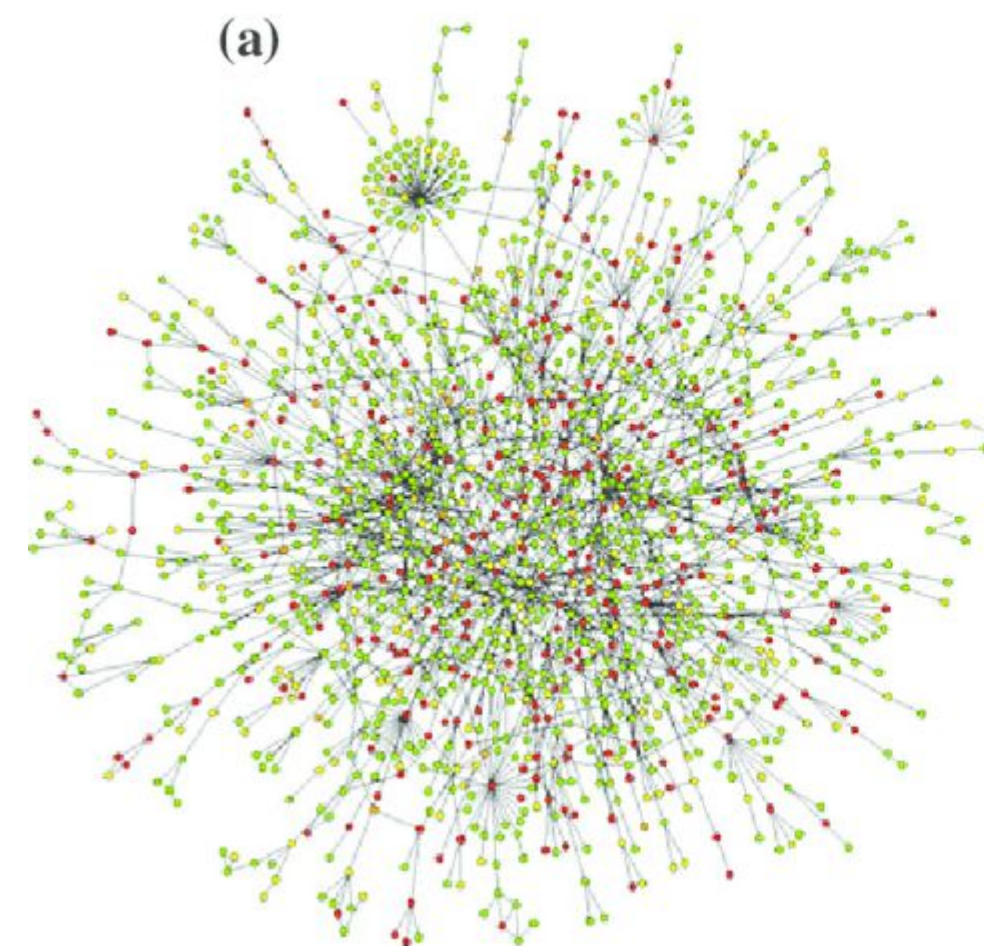
Недостаток - слабая применимость для моделирования многих процессов (из-за независимости ребер)

Модель Барабаши — Альберт

Модель генерирует так называемые безмасштабные сети - сети, вероятности степеней узлов в которых описываются степенным законом.

$$P(k) \sim k^{-\gamma}$$

Идея модели: чем больше связей вершины уже имеет, тем предпочтительнее для нее создание новой связи.



Источник графики:

https://ru.ruwiki.ru/wiki/%D0%9C%D0%BE%D0%B4%D0%B5%D0%BB%D1%8C_%D0%91%D0%B0%D1%80%D0%B0%D0%B1%D0%B0%D1%88%D0%B8_%E2%80%94%D0%90%D0%BB%D1%8C%D0%B1%D0%B5%D1%80%D1%82

Модель Барабаши — Альберт: алгоритм

Входные параметры:

- m_0 - начальное кол-во вершин
- требуемое число вершин

Генерация:

1. задаем m_0 вершин со степенью ≥ 1
2. добавляем в сеть новый узел
3. создаем новому узлу связи со старыми таким образом, что вероятно связи с каждым из них пропорциональна числу его связей

$$p_i = \frac{k_i}{\sum_j k_j}$$

Модель Барабаши — Альберт

Преимущества модели:

- Зависимость ребер друг от друга
- Конкретный обоснованный набор свойств
- Связь с процессами в реальной жизни

Недостатки:

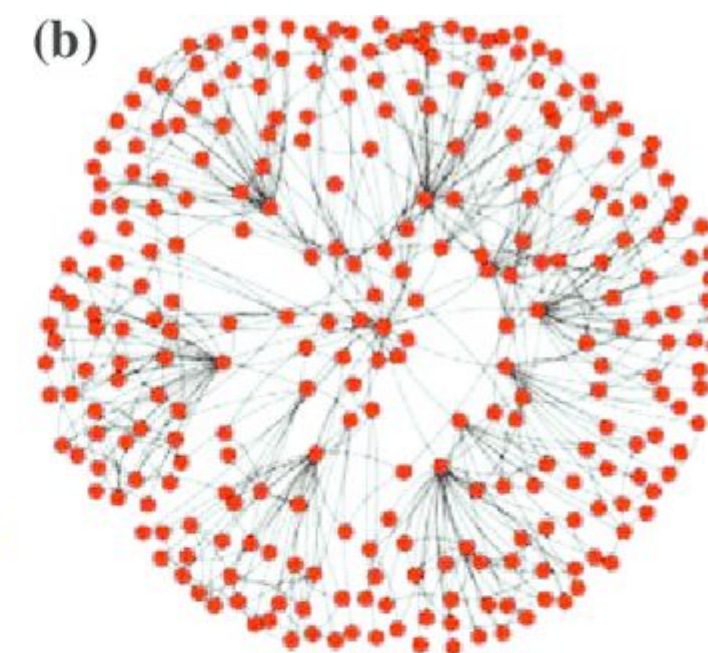
- ученые до сих пор спорят о существовании “чистых” безмасштабных сетей в реальной жизни
- итеративный алгоритм построения

Модель Уоттса-Строгаца

Генерирует графы малого мира - графы, у которых средняя длина кратчайшего пути в растёт как логарифм от количества вершин в графе.

Входные параметры:

- N - количество вершин в графе
- k - средняя степень вершины в графе, ожидается, что будет четным и будет удовлетворять условию
- p - вероятность установки связи



Источник графики:

https://ru.ruwiki.ru/wiki/%D0%9C%D0%BE%D0%B4%D0%B5%D0%BB%D1%8C_%D0%91%D0%B0%D1%80%D0%B0%D0%B1%D0%B0%D1%88%D0%B8_%E2%80%94%D0%90%D0%BB%D1%8C%D0%B1%D0%B5%D1%80%D1%82

Модель Уоттса-Строгаца

1. Строим начальный граф: N узлов по кругу, каждый из которых связан с k соседям, $k/2$ с каждой стороны
2. Нумеруем все вершины $0 \dots N - 1$
3. Для $i = 0 \dots N - 1$:
 - 3.1. Берем вершину i и ее $k/2$ крайних справа соседей.
 - 3.2. Для каждого из $k/2$ ребер перевешиваем его с вероятностью p так, чтобы не образовалось петель и кратных ребер

Модель Уоттса-Строгаца: алгоритм

Преимущества модели:

- Зависимость ребер друг от друга
- Конкретный обоснованный набор свойств
- Связь с процессами в реальной жизни (и эта связь не вызывает вопросов!)

Недостатки:

- итеративный алгоритм построения

Поиск сообществ

План:

1. Зачем это нужно
2. Girvan–Newman метод
3. Label Propagation
4. Лувенский алгоритм

Как вы считаете, зачем уметь искать сообщества в графах?

Girvan—Newman метод

Алгоритм:

1. Вычисляем степень посредничества всех ребер
2. Удаляем ребро с наибольшей степенью посредничества
3. Пересчитываем степень посредничества, где нужно
4. Повторяем шаги 2-3, пока не выполнится критерий остановки.

Плюсы:

- простой и интерпретируемый алгоритм
- иерархическая структура результата

Минус - большая вычислительная сложность.

Label Propagation

Алгоритм:

1. Каждой вершине присваивается своя метка кластера
2. Перебираем вершины в случайном порядке. Каждой присваиваем метку, которая чаще всего встречается среди соседей. Если таких меток несколько, то выбираем случайным, но последовательным образом одну из них.
3. Повторяем пункт 2, либо пока алгоритм не сойдется, либо пока не будет достигнут максимум итераций.

Плюсы:

- можно манипулировать начальными условиями и способом вычисления доминирующей метки
- дешевые вычисления

Лувенский алгоритм

Алгоритм:

1. Каждой вершине присваивается своя метка кластера
2. Перебираем вершины. Для каждой считаем прибыль по модулярности (плотность относительно случайного графа) при добавлении ее в каждый из кластеров. Добавляем в тот, где значение наибольшее.
3. Сжимаем кластеры до одной вершины.
4. Повторяем пункты 2-3, либо пока алгоритм не сойдется, либо пока не будет достигнут максимум итераций.

